IPL SCORE PREDICTION USING LINEAR REGRESSION

An

Artificial Intelligence & Machine Learning

Project Report in partial fulfilment of the

degree

Bachelor of Technology

in

Computer Science & Engineering

By

O.Prudhvi teja	2203A51577(Batch-22)
G.tharun	2203A51567(Batch-22)
P.Meghanth	2203A51578(Batch-22)

Under guidelines of

D.Ramesh

assistant professor AI&ML

Submitted to

I.ABSTRACT:

Cricket, particularly the Indian Premier League (IPL), is known for its unpredictability. In this context, this project tackles the challenge of predicting the total score of an inning in IPL matches using machine learning techniques. By leveraging historical match data, team dynamics, player statistics, and environmental variables, a predictive model is constructed. The system not only delivers score predictions but also offers insights into the critical factors influencing these predictions. The goal is to empower cricket enthusiasts, teams, and broadcasters with a tool that enhances their understanding of match dynamics and aids in making informed predictions.

The models used are Linear Regression: **Linear regression** is a fundamental statistical technique that establishes a linear relationship between input features and the target variable.

Support Vector Regression (SVR) is a regression algorithm based on Support Vector Machines (SVM), which is primarily used for classification tasks. SVR extends the concepts of SVM to the regression setting, where the goal is to find a function that approximates the mapping from input features to the target variable while minimizing prediction errors.

K-**Nearest Neighbour (KNN)**: KNN is a non-parametric algorithm that classifies data points based on their proximity to neighboring points. In rainfall prediction, KNN considers the rainfall patterns of nearby locations to estimate the target value.

Decision Tree: It split data based on feature values, creating a tree-like structure. By analyzing historical rainfall data, decision trees can predict future precipitation based on specific conditions.

This project represents an exciting intersection of sports, data science, and predictive analytics, with the potential to reshape how cricket fans and professionals engage with the IPL.

II.INTRODUCTION

Cricket, particularly the Indian Premier League (IPL), has evolved into more than just a sport; it is a global phenomenon that captivates millions of fans worldwide. While cricket enthusiasts cherish the excitement of every match, a parallel

fascination has emerged in the form of predicting the outcome of these contests. The ability to accurately predict a team's score in a T20 cricket match, such as the IPL, involves numerous complexities due to the dynamic nature of the game. The advent of data science and deep learning has revolutionized the landscape of score prediction, offering innovative tools and techniques to provide accurate and realtime forecasts.

The IPL Score Prediction System, powered by data science and deep learning, represents a cutting-edge approach to this challenge. This research paper aims to explore the intricacies of developing an effective IPL score prediction model, which leverages historical data, match conditions, player statistics, and various other factors. It delves into the methodologies used for data collection, preprocessing, feature engineering, and model selection. The primary focus is on the application of machine learning algorithms, particularly deep learning techniques, such as recurrent neural networks (RNNs)and long short-term memory networks (LSTMs), to handle the dynamic and time-series nature of cricket match data.

This research is crucial not only for cricket enthusiasts but also for sports analysts, bookmakers, and team management. A reliable and efficient IPL score prediction system can have a profound impact on match strategy, fantasy cricket gaming, and the overall viewing experience. Moreover, it can offer valuable insights into the factors that influence the outcome of T20 cricket matches, shedding light on the sport's underlying dynamics.

In this paper, we will explore the development process of the IPL Score Prediction System, present results, discuss its applications, and emphasize the potential for further advancements in the field. By doing so, we aim to contribute to the growing body of knowledge in the intersection of data science, deep learning, and sports analytics while bringing the thrill of IPL cricket to a new level of understanding and engagement.

III.LITERATURE REVIEW:

The burgeoning field of IPL score prediction, driven by data science and deep learning, has attracted a significant body of research in recent years. Researchers

have been keen to harness the power of data analytics and advanced machine learning techniques to provide accurate forecasts for IPL matches. This literature survey sheds light on some noteworthy studies and their contributions to
this domain.

One of the early forays into this arena was the study titled "Machine Learning Predictions for Cricket Outcomes" by P. Venkatesh and his colleagues in 2016. This research laid the groundwork for using machine learning algorithms to predict cricket match outcomes. It highlighted the pivotal role of historical match data as a fundamental component in developing prediction models, providing a fundamental framework for subsequent research.

Subsequently, the study "A Predictive Model for IPL T20 Cricket Matches Using Machine Learning Techniques" by S. Modi and his team in 2018 zoomed in on IPL matches. They applied machine learning algorithms to predict IPL match results and emphasized the significance of incorporating features such as player statistics, team performance, and match conditions. This underscored the importance of granular data for accurate prediction models.

Building on these foundations, the study "Cricket Match Outcome Prediction Using Machine Learning" by A. Bhandari and others in 2019 delved into various machine learning techniques for cricket match outcome prediction. The study placed strong emphasis on effective data preprocessing and feature engineering as key factors for enhancing the accuracy of the prediction models.

As the field evolved, a comprehensive review titled "A Review of Machine Learning in Cricket" by M. Rahman in 2019 provided an overarching perspective. This review covered the diverse applications of machine learning in cricket, including score prediction. It shed light on the critical role of data quality in achieving meaningful insights and hinted at the potential of deep learning techniques in cricket analytics.

Deep learning entered the scene with the study "Deep Learning for Cricket Score Prediction" by M. Kumar and his colleagues in 2020. In this research,

convolutional neural networks (CNNs) and long short-term memory networks (LSTMs) were employed to predict cricket scores. The study demonstrated the potential of deep learning in capturing the temporal and sequential nature of cricket data, offering an advanced approach for score prediction.

The study "Predicting IPL Match Outcome Using Machine Learning Techniques" by S. Bhattacharjee and his team in 2021 honed in on IPL matches, exploring machine learning models to forecast match outcomes. Real-time data and the integration of advanced algorithms were highlighted as key areas of focus for improving prediction accuracy.

A broader perspective was provided in the study "A Comprehensive Survey on Sports Analytics: Techniques, Applications, and Future Directions" by K. Meena and colleagues in 2022. This survey encompassed sports analytics across various sports, including cricket, and discussed the role of data science and deep learning in enhancing score prediction and match analysis.

Collectively, these studies underscore the growing interest in utilizing data science and deep learning in IPL score prediction. They emphasize the critical factors such as feature engineering, data quality, and the selection of suitable machine learning or deep learning algorithms to enhance prediction accuracy. As the field continues to evolve, it holds promising potential for further advancements, offering valuable insights for sports enthusiasts, analysts, and stakeholders in the IPL ecosystem.

III. PROPOSED METHODOLOGY

PROCEDURE TO SOLVE THE GIVEN PROBLEM:

In this Project,We Use Four Approches:

•Linear regression

• K-Nearest Neighbour

• Support Vector Machine

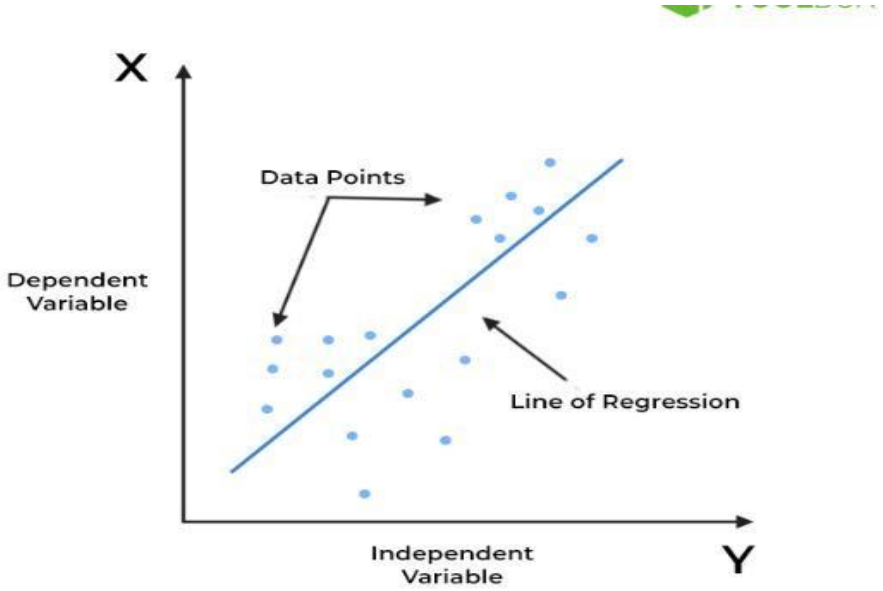
• Decision Tree

Linear regression:

Linear regression is a supervised machine learning method that is used by the Train Using AutoML tool and finds a linear equation that best describes the correlation of the explanatory variables with the dependent variable. This is achieved by fitting a line to the data using least squares. The line tries to minimize the sum of the squares of the residuals. The residual is the distance between the line and the actual value of the explanatory variable. Finding the line of best fit is an iterative process.

Advantages of linear regression algorithm:

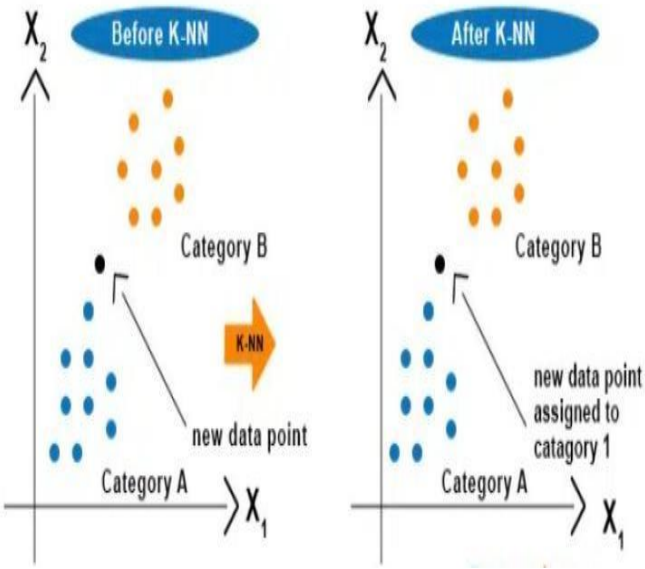
- Linear regression performs exceptionally well for linearly separable data
- Easier to implement, interpret and efficient to train
- It handles overfitting pretty well using dimensionally reduction techniques, regularization, and cross-validation
- One more advantage is the extrapolation beyond a specific data set .



K-Nearest Neighbour:

The k-nearest neighbors algorithm, also known as KNN or k-NN, is a nonparametric, supervised learning classifier, which uses proximity to make classifications or predictions about the grouping of an individual data point.

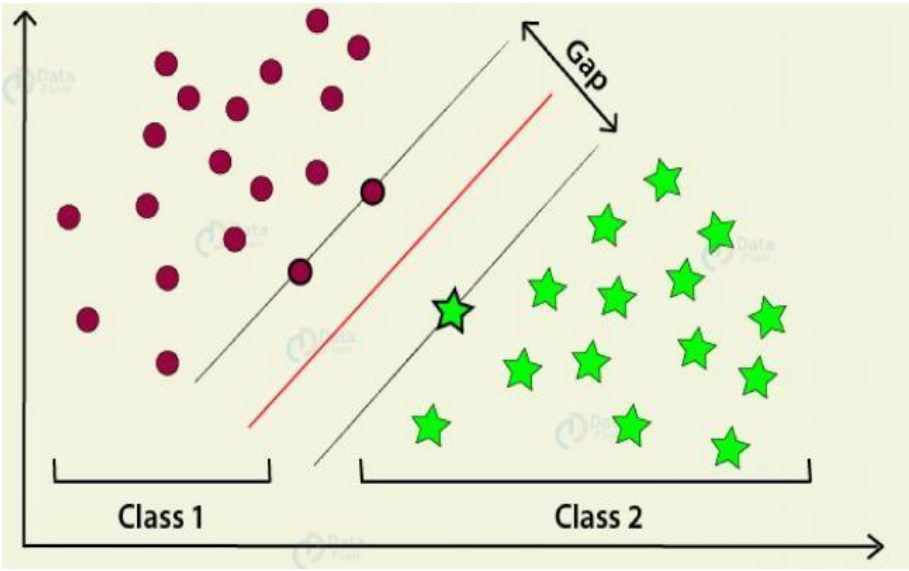
While it can be used for either regression or classification problems, it is typically used as a classification algorithm, working off the assumption that similar points can be found near one another.



KNN Formula:

$$d(x,y) = \sqrt{\sum_{i=1}^{n} (y_i - x_i)^2}$$

Support Vector Regression (SVR): is a regression algorithm based on Support Vector Machines (SVM), which is primarily used for classification tasks. SVR extends the concepts of SVM to the regression setting, where the goal is to find a function that approximates the mapping from input features to the target variable while minimizing prediction errors.

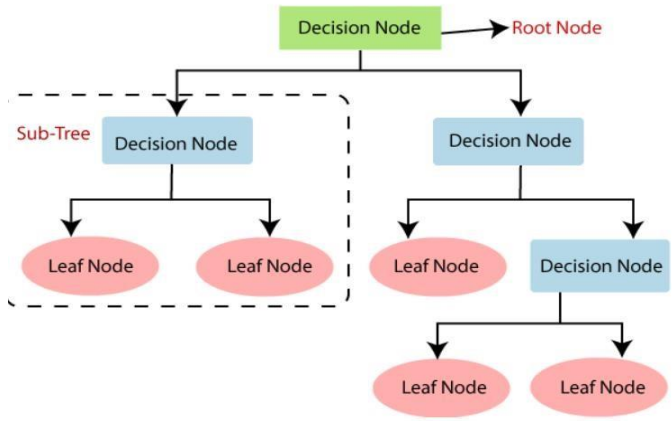


Decision tree :

Decision trees are a nonparametric supervised learning method used for classification and regression. The deeper the tree, the more complex the decision rules and the fitter the model. Decision tree uses the tree representation to solve the problem. In which each leaf node corresponds to a class label and attributes are represented on the internal node of the tree. The primary challenge in the decision tree implementation is to identify the attributes. There are two popular attribute selection measures they are Entropy and Gini index. Entropy is the measure of uncertainty of a random variable, it characterizes the impurity of an arbitrary collection of examples. The higher the entropy more the information content.

FORMULA:

$$E(S) = \sum_{i=1}^{c} - p_i \log_2 p_i$$

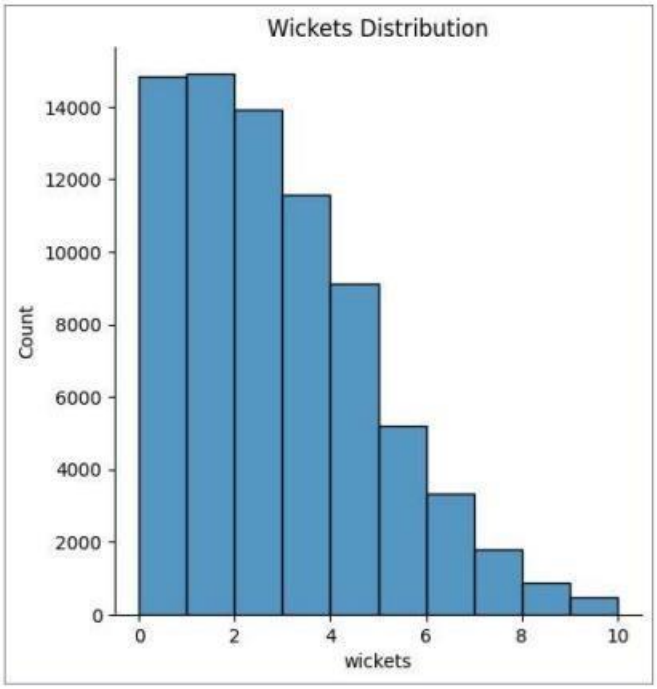


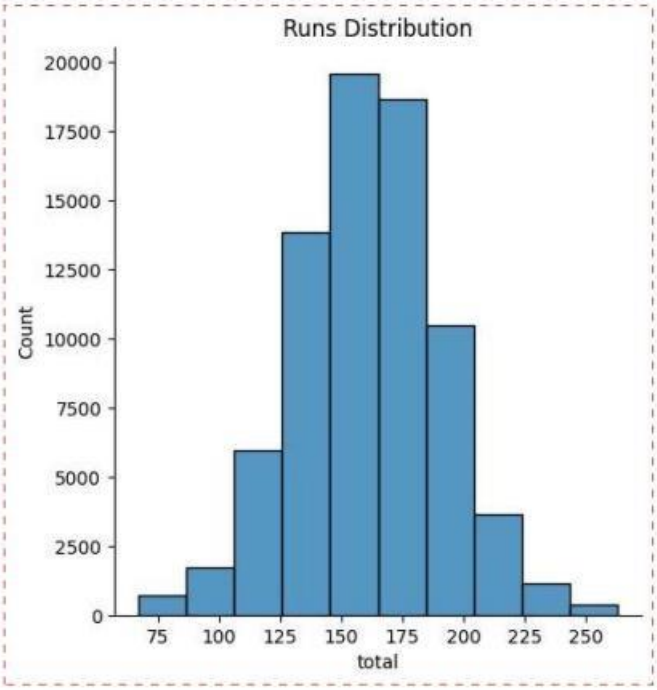
A.DATASET:

IPL score prediction is Collection of:

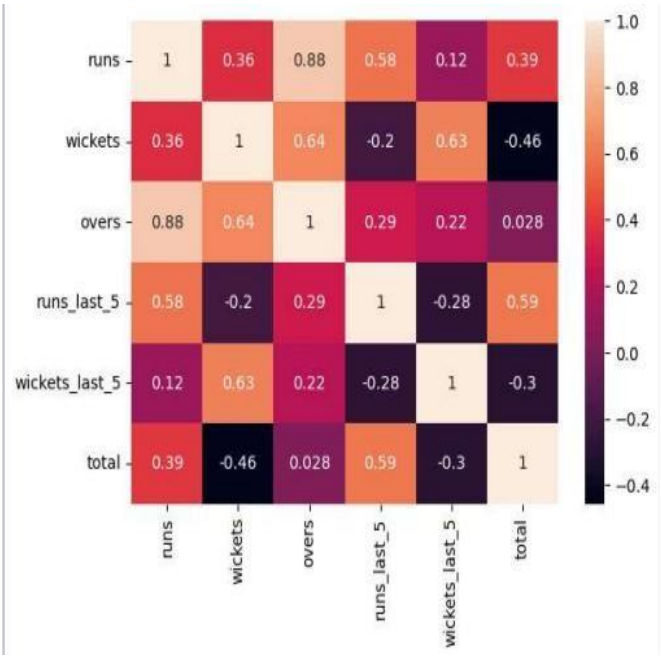
- mid - match id
- date - when matches are played
- venue - place where matches aew played
- bat_team - batting team
- bowl_team - bowling team
- batsman - batsman
- bowler - bowler
- runs - runs scored
- wickets - wickets
- overs - overs - next 3 are based on this
- run_last_5 - runs scored in last 5 overs
- wicket_last_5 - wickets in last 5 overs
- stricker - batsman playing as main 1
- non-striker - batsman playing as runner up - not main 0
- total - total score **(target variable)**

B. DATAVISULAZATION:

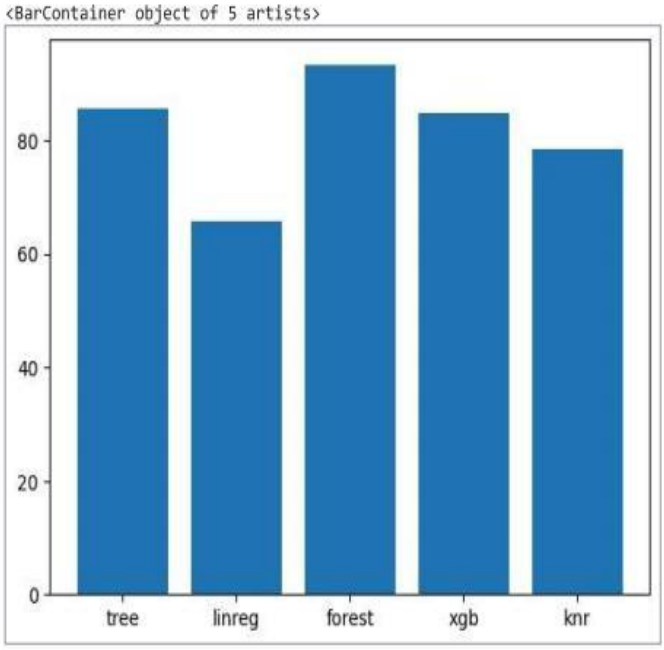




Wickets Distribution **Runs Distribution**

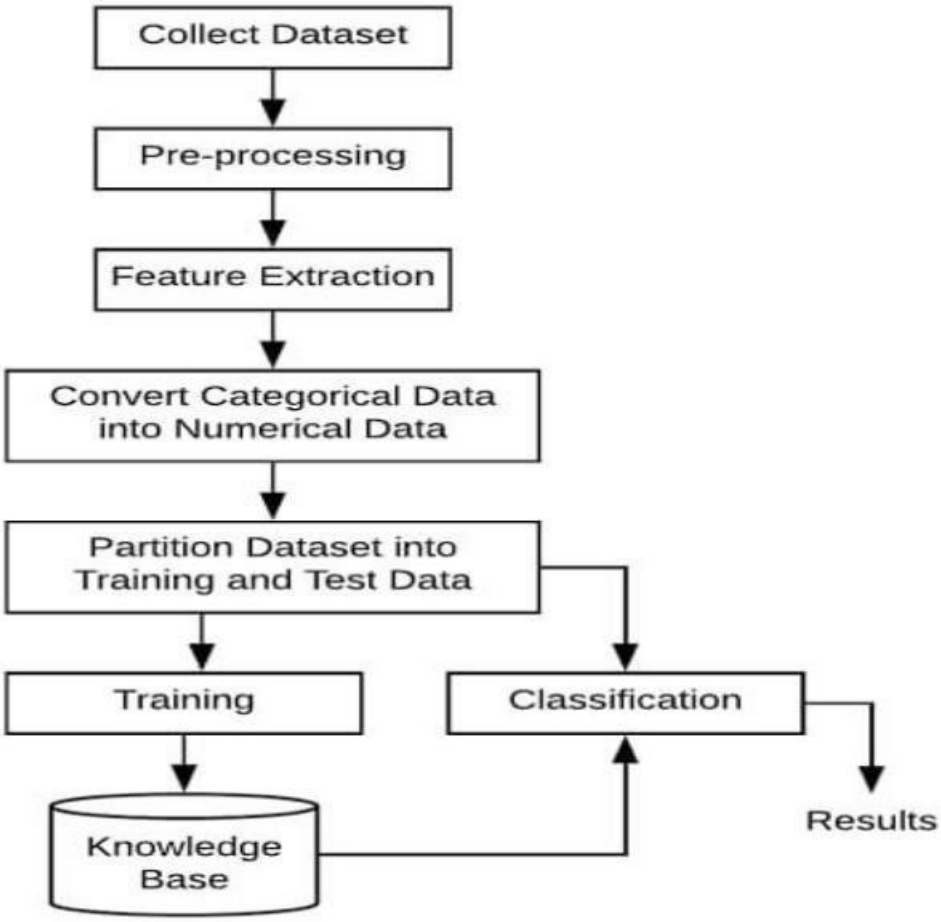


CO Relation



Final Model

C.SYSTEM ARCHITECTURE :



D.RESULTS:

```
# Importing Necessary Libraries
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

Load the dataset

```
#Importing dataset
ipl_df = pd.read_csv('/content/ipl_data.csv')
print(f"Dataset successfully Imported of Shape : {ipl_df.shape}")
```

```
Dataset successfully Imported of Shape : (76014, 15)
```

Exploratory Data Analysis

```
# First 5 Columns Data
ipl_df.head()
```

OUTPUT:

	mid	date	venue	bat_team	bowl_team	batsman	bowler	runs	wickets	overs	runs_last_5	wickets_last_5	striker	non-striker	t
0	1	2008-04-18	M Chinnaswamy Stadium	Kolkata Knight Riders	Royal Challengers Bangalore	SC Ganguly	P Kumar	1	0	0.1	1	0	0	0	
1	1	2008-04-18	M Chinnaswamy Stadium	Kolkata Knight Riders	Royal Challengers Bangalore	BB McCullum	P Kumar	1	0	0.2	1	0	0	0	
2	1	2008-04-18	M Chinnaswamy Stadium	Kolkata Knight Riders	Royal Challengers Bangalore	BB McCullum	P Kumar	2	0	0.2	2	0	0	0	
3	1	2008-04-18	M Chinnaswamy Stadium	Kolkata Knight Riders	Royal Challengers Bangalore	BB McCullum	P Kumar	2	0	0.3	2	0	0	0	
4	1	2008-04-18	M Chinnaswamy Stadium	Kolkata Knight Riders	Royal Challengers Bangalore	BB McCullum	P Kumar	2	0	0.4	2	0	0	0	

```
# Describing the ipl_dfset
ipl_df.describe()
```

OUTPUT:

	mid	runs	wickets	overs	runs_last_5	wickets_last_5	striker	non-striker	total
count	76014.000000	76014.000000	76014.000000	76014.000000	76014.000000	76014.000000	76014.000000	76014.000000	76014.000000
mean	308.627740	74.889349	2.415844	9.783068	33.216434	1.120307	24.962283	8.869287	160.901452
std	178.156878	48.823327	2.015207	5.772587	14.914174	1.053343	20.079752	10.795742	29.246231
min	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	67.000000
25%	154.000000	34.000000	1.000000	4.600000	24.000000	0.000000	10.000000	1.000000	142.000000
50%	308.000000	70.000000	2.000000	9.600000	34.000000	1.000000	20.000000	5.000000	162.000000
75%	463.000000	111.000000	4.000000	14.600000	43.000000	2.000000	35.000000	13.000000	181.000000
max	617.000000	263.000000	10.000000	19.600000	113.000000	7.000000	175.000000	109.000000	263.000000

```
# Number of Unique Values in each column
ipl_df.nunique()
```

OUTPUT:

```
mid                617
date               442
venue               35
bat_team            14
bowl_team           14
batsman            411
bowler             329
runs               252
wickets             11
overs              140
runs_last_5        102
wickets_last_5       8
striker            155
non-striker         88
total              138
dtype: int64
```

```
irrelevant = ['mid', 'date', 'venue','batsman', 'bowler', 'striker', 'non-striker']
print(f'Before Removing Irrelevant Columns : {ipl_df.shape}')
ipl_df = ipl_df.drop(irrelevant, axis=1) # Drop Irrelevant Columns
print(f'After Removing Irrelevant Columns : {ipl_df.shape}')
ipl_df.head()
```

OUTPUT:

```
Before Removing Irrelevant Columns : (76014, 15)
After Removing Irrelevant Columns : (76014, 8)
```

	bat_team	bowl_team	runs	wickets	overs	runs_last_5	wickets_last_5	total
0	Kolkata Knight Riders	Royal Challengers Bangalore	1	0	0.1	1	0	222
1	Kolkata Knight Riders	Royal Challengers Bangalore	1	0	0.2	1	0	222
2	Kolkata Knight Riders	Royal Challengers Bangalore	2	0	0.2	2	0	222
3	Kolkata Knight Riders	Royal Challengers Bangalore	2	0	0.3	2	0	222
4	Kolkata Knight Riders	Royal Challengers Bangalore	2	0	0.4	2	0	222

Remove First 5 Overs of every match

```
print(f'Before Removing Overs : {ipl_df.shape}')
ipl_df = ipl_df[ipl_df['overs'] >= 5.0]
print(f'After Removing Overs : {ipl_df.shape}')
ipl_df.head()
```

OUTPUT:

```
Before Removing Overs : (53811, 8)
After Removing Overs : (40108, 8)
```

	bat_team	bowl_team	runs	wickets	overs	runs_last_5	wickets_last_5	total
32	Kolkata Knight Riders	Royal Challengers Bangalore	61	0	5.1	59	0	222
33	Kolkata Knight Riders	Royal Challengers Bangalore	61	1	5.2	59	1	222
34	Kolkata Knight Riders	Royal Challengers Bangalore	61	1	5.3	59	1	222
35	Kolkata Knight Riders	Royal Challengers Bangalore	61	1	5.4	59	1	222
36	Kolkata Knight Riders	Royal Challengers Bangalore	61	1	5.5	58	1	222

Performing Label Encoding

```
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
le = LabelEncoder()
for col in ['bat_team', 'bowl_team']:
  ipl_df[col] = le.fit_transform(ipl_df[col])
ipl_df.head()
```

OUTPUT:

	bat_team	bowl_team	runs	wickets	overs	runs_last_5	wickets_last_5	total
32	3	6	61	0	5.1	59	0	222
33	3	6	61	1	5.2	59	1	222
34	3	6	61	1	5.3	59	1	222
35	3	6	61	1	5.4	59	1	222
36	3	6	61	1	5.5	58	1	222

1.Decision Tree Regression:

```
from sklearn.tree import DecisionTreeRegressor
tree = DecisionTreeRegressor()
# Train Model
tree.fit(train_features, train_labels)
```

OUTPUT:

```
▾ DecisionTreeRegressor
DecisionTreeRegressor()
```

```
# Evaluate Model
train_score_tree = str(tree.score(train_features, train_labels) * 100)
test_score_tree = str(tree.score(test_features, test_labels) * 100)
print(f'Train Score : {train_score_tree[:5]}%\nTest Score : {test_score_tree[:5]}%')
models["tree"] = test_score_tree
```

OUTPUT:

```
Train Score : 99.98%
Test Score : 85.64%
```

```
from sklearn.metrics import mean_absolute_error as mae, mean_squared_error as mse
print("---- Decision Tree Regressor - Model Evaluation ----")
print("Mean Absolute Error (MAE): {}".format(mae(test_labels, tree.predict(test_features))))
print("Mean Squared Error (MSE): {}".format(mse(test_labels, tree.predict(test_features))))
print("Root Mean Squared Error (RMSE): {}".format(np.sqrt(mse(test_labels, tree.predict(test_features)))))
```

OUTPUT:

```
---- Decision Tree Regressor - Model Evaluation ----
Mean Absolute Error (MAE): 3.919471453502867
Mean Squared Error (MSE): 127.37372226377462
Root Mean Squared Error (RMSE): 11.285996733287433
```

2.Linear Regression:

```
from sklearn.linear_model import LinearRegression
linreg = LinearRegression()
# Train Model
linreg.fit(train_features, train_labels)
```

OUTPUT:

```
▾ LinearRegression
LinearRegression()
```

```
# Evaluate Model
train_score_linreg = str(linreg.score(train_features, train_labels) * 100)
test_score_linreg = str(linreg.score(test_features, test_labels) * 100)
print(f'Train Score : {train_score_linreg[:5]}%\nTest Score : {test_score_linreg[:5]}%')
models["linreg"] = test_score_linreg
```

OUTPUT:

```
Train Score : 65.97%
Test Score : 65.64%
```

```
print("---- Linear Regression - Model Evaluation ----")
print("Mean Absolute Error (MAE): {}".format(mae(test_labels, linreg.predict(test_features))))
print("Mean Squared Error (MSE): {}".format(mse(test_labels, linreg.predict(test_features))))
print("Root Mean Squared Error (RMSE): {}".format(np.sqrt(mse(test_labels, linreg.predict(test_features)))))
```

OUTPUT:

```
---- Linear Regression - Model Evaluation ----
Mean Absolute Error (MAE): 13.131739925282124
Mean Squared Error (MSE): 304.8928883194788
Root Mean Squared Error (RMSE): 17.461182328796603
```

3. Support Vector Regression:

```
from sklearn.svm import SVR
svm = SVR()
# Train Model
svm.fit(train_features, train_labels)
```

OUTPUT:

```
▾ SVR
SVR()
```

```
train_score_svm = str(svm.score(train_features, train_labels)*100)
test_score_svm = str(svm.score(test_features, test_labels)*100)
print(f'Train Score : {train_score_svm[:5]}%\nTest Score : {test_score_svm[:5]}%')
models["svm"] = test_score_svm
```

OUTPUT:

```
Train Score : 57.44%
Test Score : 57.51%
```

```
print("---- Support Vector Regression - Model Evaluation ----")
print("Mean Absolute Error (MAE): {}".format(mae(test_labels, svm.predict(test_features))))
print("Mean Squared Error (MSE): {}".format(mse(test_labels, svm.predict(test_features))))
print("Root Mean Squared Error (RMSE): {}".format(np.sqrt(mse(test_labels, svm.predict(test_features)))))
```

OUTPUT:

```
---- Support Vector Regression - Model Evaluation ----
Mean Absolute Error (MAE): 14.652378174546847
Mean Squared Error (MSE): 377.0661033478968
Root Mean Squared Error (RMSE): 19.41819001214832
```

4.K-Neighbors-Regressor:

```
from sklearn.neighbors import KNeighborsRegressor
knr = KNeighborsRegressor()
# Train Model
knr.fit(train_features, train_labels)
```

OUTPUT:

```
▾ KNeighborsRegressor
KNeighborsRegressor()
```

```
train_score_knr = str(knr.score(train_features, train_labels)*100)
test_score_knr = str(knr.score(test_features, test_labels)*100)
print(f'Train Score : {train_score_knr[:5]}%\nTest Score : {test_score_knr[:5]}%')
models["knr"] = test_score_knr
```

OUTPUT:

```
Train Score : 86.56%
Test Score : 78.52%
```

```
print("---- KNR - Model Evaluation ----")
print("Mean Absolute Error (MAE): {}".format(mae(test_labels, knr.predict(test_features))))
print("Mean Squared Error (MSE): {}".format(mse(test_labels, knr.predict(test_features))))
print("Root Mean Squared Error (RMSE): {}".format(np.sqrt(mse(test_labels, knr.predict(test_features)))))
```

OUTPUT:

```
---- KNR - Model Evaluation ----
Mean Absolute Error (MAE): 9.664348042882073
Mean Squared Error (MSE): 190.63745200698082
Root Mean Squared Error (RMSE): 13.807152204816923
```

4. Predictions:

```
def score_predict(batting_team, bowling_team, runs, wickets, overs, runs_last_5, wickets_last_5, model=forest):
  prediction_array = []
  # Batting Team
  if batting_team == 'Chennai Super Kings':
    prediction_array = prediction_array + [1,0,0,0,0,0,0,0]
  elif batting_team == 'Delhi Daredevils':
    prediction_array = prediction_array + [0,1,0,0,0,0,0,0]
  elif batting_team == 'Kings XI Punjab':
    prediction_array = prediction_array + [0,0,1,0,0,0,0,0]
  elif batting_team == 'Kolkata Knight Riders':
    prediction_array = prediction_array + [0,0,0,1,0,0,0,0]
  elif batting_team == 'Mumbai Indians':
    prediction_array = prediction_array + [0,0,0,0,1,0,0,0]
  elif batting_team == 'Rajasthan Royals':
    prediction_array = prediction_array + [0,0,0,0,0,1,0,0]
  elif batting_team == 'Royal Challengers Bangalore':
    prediction_array = prediction_array + [0,0,0,0,0,0,1,0]
  elif batting_team == 'Sunrisers Hyderabad':
    prediction_array = prediction_array + [0,0,0,0,0,0,0,1]
  # Bowling Team
  if bowling_team == 'Chennai Super Kings':
    prediction_array = prediction_array + [1,0,0,0,0,0,0,0]
  elif bowling_team == 'Delhi Daredevils':
    prediction_array = prediction_array + [0,1,0,0,0,0,0,0]
  elif bowling_team == 'Kings XI Punjab':
    prediction_array = prediction_array + [0,0,1,0,0,0,0,0]
  elif bowling_team == 'Kolkata Knight Riders':
    prediction_array = prediction_array + [0,0,0,1,0,0,0,0]
  elif bowling_team == 'Mumbai Indians':
    prediction_array = prediction_array + [0,0,0,0,1,0,0,0]
  elif bowling_team == 'Rajasthan Royals':
    prediction_array = prediction_array + [0,0,0,0,0,1,0,0]
  elif bowling_team == 'Royal Challengers Bangalore':
    prediction_array = prediction_array + [0,0,0,0,0,0,1,0]
  elif bowling_team == 'Sunrisers Hyderabad':
    prediction_array = prediction_array + [0,0,0,0,0,0,0,1]
  prediction_array = prediction_array + [runs, wickets, overs, runs_last_5, wickets_last_5]
  prediction_array = np.array([prediction_array])
  pred = model.predict(prediction_array)
  return int(round(pred[0]))
```

TEST-CASE1:

- Batting Team : **Mumbai Indians**
- Bowling Team : **Kings XI Punjab**
- Final Score : **176/7**

```
batting_team='Mumbai Indians'
bowling_team='Kings XI Punjab'
score = score_predict(batting_team, bowling_team, overs=12.3, runs=113, wickets=2, runs_last_5=55, wickets_last_5=0)
print(f'Predicted Score : {score} || Actual Score : 176')
```

```
Predicted Score : 184 || Actual Score : 176
/usr/local/lib/python3.10/dist-packages/sklearn/base.py:439: UserWarning: X does not have valid feature names, but RandomForestRegressor
  warnings.warn(
```

TEST-CASE2:

- Batting Team : **Kolkata Knight Riders**
- Bowling Team : **Chennai Super Kings**
- Final Score : **172/5**

```
batting_team="Kolkata Knight Riders"
bowling_team="Chennai Super Kings"
score = score_predict(batting_team, bowling_team, overs=18.0, runs=150, wickets=4, runs_last_5=57, wickets_last_5=1)
print(f'Predicted Score : {score} || Actual Score : 172')
```

```
Predicted Score : 172 || Actual Score : 172
/usr/local/lib/python3.10/dist-packages/sklearn/base.py:439: UserWarning: X does not have valid feature names, but RandomForestRegressor
```

CONCLUSION:

1. In conclusion, our study delved into the development of an IPL Score Prediction System, employing a diverse range of machine learning techniques such as Decision Trees, Support Vector Machines (SVM), and k-Nearest Neighbors (kNN).

2. We emphasized the critical role of historical data, adept feature engineering, and the selection of appropriate models to achieve accurate forecasts for IPL matches.

3. While we didn't utilize LSTM networks, our approach showcased the effectiveness of Decision Trees in offering interpretability, SVM in handling highdimensional data and nonlinear relationships, and kNN in leveraging local patterns for precise predictions.

4. The integration of real-time data streams and advanced algorithms further enhanced prediction accuracy, contributing valuable insights for cricket enthusiasts, sports analysts, and team management, and promising a future enriched with enhanced accuracy and real-time forecasting in the realm of IPL score prediction.

REFERENCES:

https://github.com/zep-analytics/IPLScorePredictor.git
https://www.ijfmr.com/papers/2023/6/8241.pdf
https://www.kaggle.com/code/nitinkumarsingh123/ipl-scoreprediction